

Análise Organizacional e Soluções Tecnológicas

Prof. Me. Rafael Gonçalves de
Oliveira Viana

Módulo 3 - Desenvolvimento da Solução

Unidade 1 – Identificação de Arquitetura/Tecnologias e Ferramentas

O que será abordado

- Projeto de Arquitetura de Software.
- Estilos Arquiteturais.
- Frameworks/Tecnologias Relevantes.

Projeto de Arquitetura de Software

Conceito segundo Sommerville (2018, p.148)

"Projeto de arquitetura visa compreender como um sistema de software deve ser organizado e projetar a estrutura geral desse sistema"

Estilos Arquiteturais

- Arquitetura em camadas.
- Arquitetura em repositório.
- Arquitetura em cliente-servidor.
- Arquitetura em duto e filtro.

Arquitetura em Camada

- **Descrição**

- Sistema organizado em camadas hierárquicas, cada uma com funcionalidades específicas;
- Cada camada fornece serviços para a camada superior;
- Camadas inferiores oferecem serviços essenciais compartilhados por todo o sistema.

Arquitetura em Camada

- Quando utilizar
 - Ao adicionar novos recursos sobre sistemas existentes;
 - Quando o desenvolvimento é dividido entre diferentes equipes, cada uma focada em uma camada;
 - Quando há necessidade de segurança em múltiplos níveis.

Arquitetura em Camada

- **Vantagens**

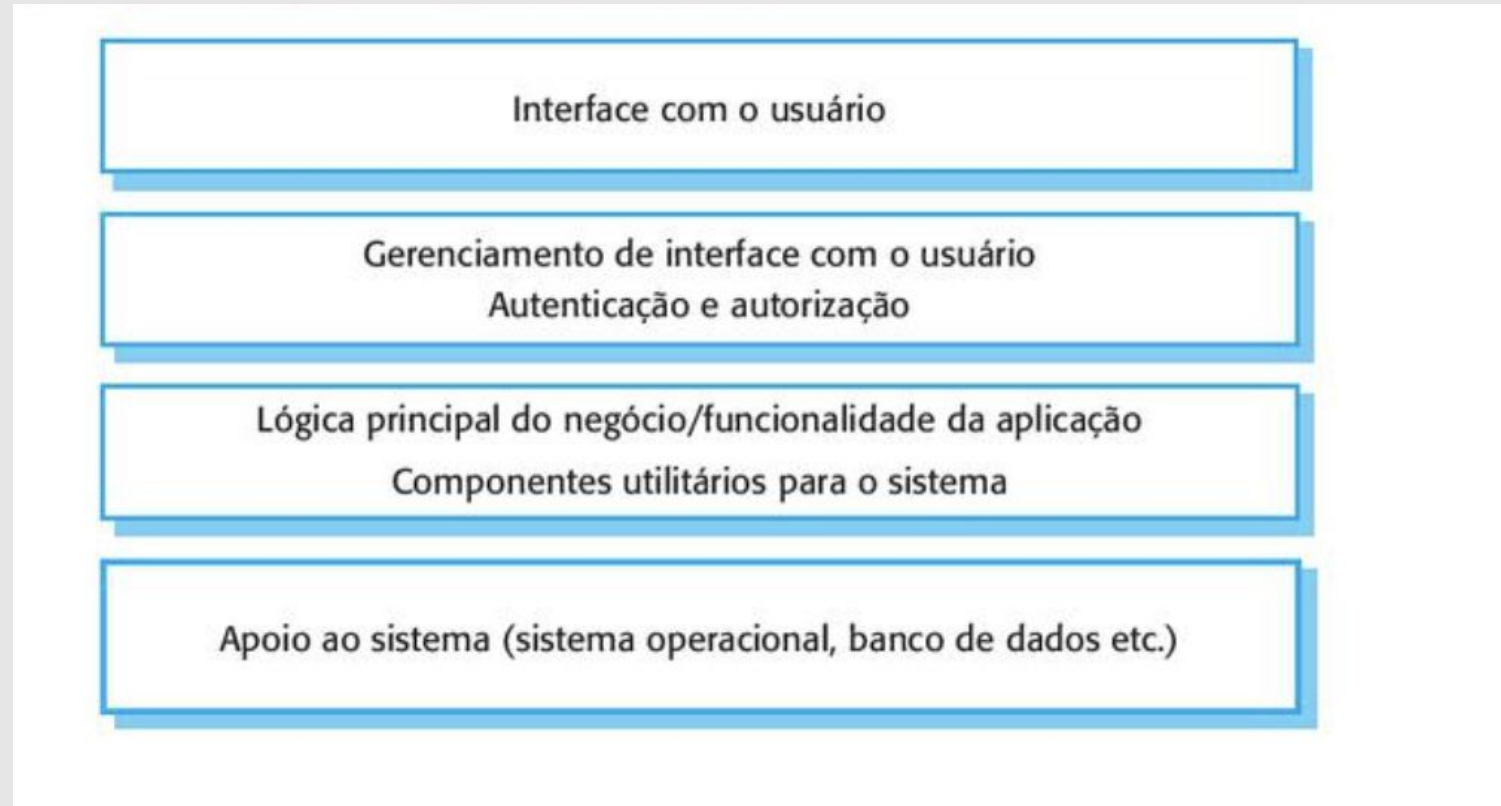
- Camadas podem ser substituídas desde que mantenham a mesma interface;
- Funcionalidades redundantes (ex.: autenticação) podem estar presentes em várias camadas;
- Favorece manutenção e evolução incremental.

Arquitetura em Camada

- **Desvantagens**

- Separação nem sempre clara entre camadas;
- Interações diretas entre camadas distantes podem ser necessárias;
- Desempenho prejudicado por múltiplos níveis de interpretação de requisições.

Arquitetura em Camada



Fonte: Sommerville, 2018, pág.158.

Arquitetura de Repositório

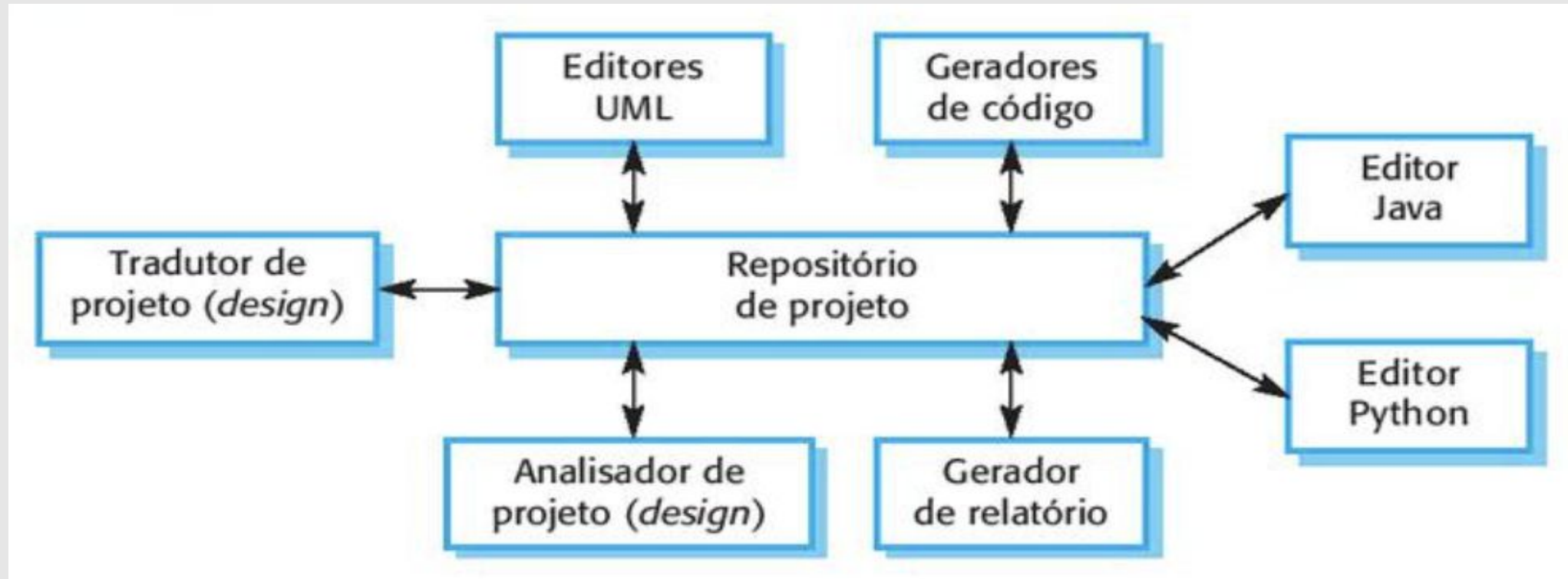
- **Descrição**

- Os dados do sistema ficam em um repositório central.
- Componentes não interagem diretamente, apenas por meio do repositório.

- **Quando utilizar**

- Volume de dados a serem armazenados por longos períodos.
- Sistemas dirigidos por dados, em que atualizações disparam ações ou ferramentas.

Arquitetura em Repositório



Fonte: Sommerville, 2018, pág.160.

Arquitetura em Cliente-Servidor

- **Descrição**

- Sistema dividido entre clientes e servidores;
- Cada serviço é executado em um servidor separado.

- **Quando utilizar**

- Quando há necessidade de acesso a dados compartilhados de diferentes locais;
- Quando há variação de carga (servidores podem ser replicados).

Arquitetura em Cliente-Servidor

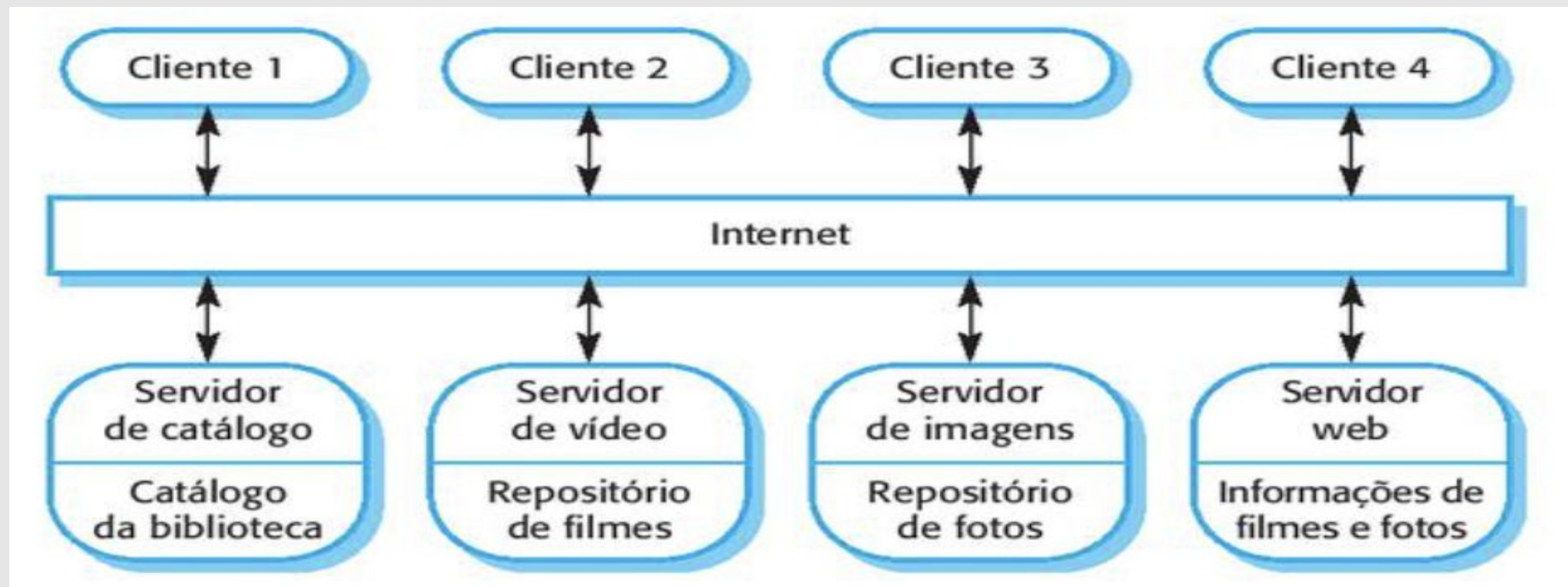
- **Vantagens**

- Servidores podem ser distribuídos em rede;
- Funcionalidade centralizada acessível a todos os clientes;
- Reduz funcionalidades em cada cliente.

- **Desvantagens**

- Cada serviço é um ponto único de falha;
- Desempenho variável, depende da rede e do servidor.

Arquitetura de Cliente-Servidor



Fonte: Sommerville, 2018, pág.162.

Arquitetura em Duto e Filtro

- **Descrição**

- Os dados fluem (como em um duto) de um componente para outro para serem processados.

- **Quando é utilizado**

- Entradas são processadas em estágios separados, gerando saídas relacionadas.

Arquitetura em Duto e Filtro

- **Vantagens**

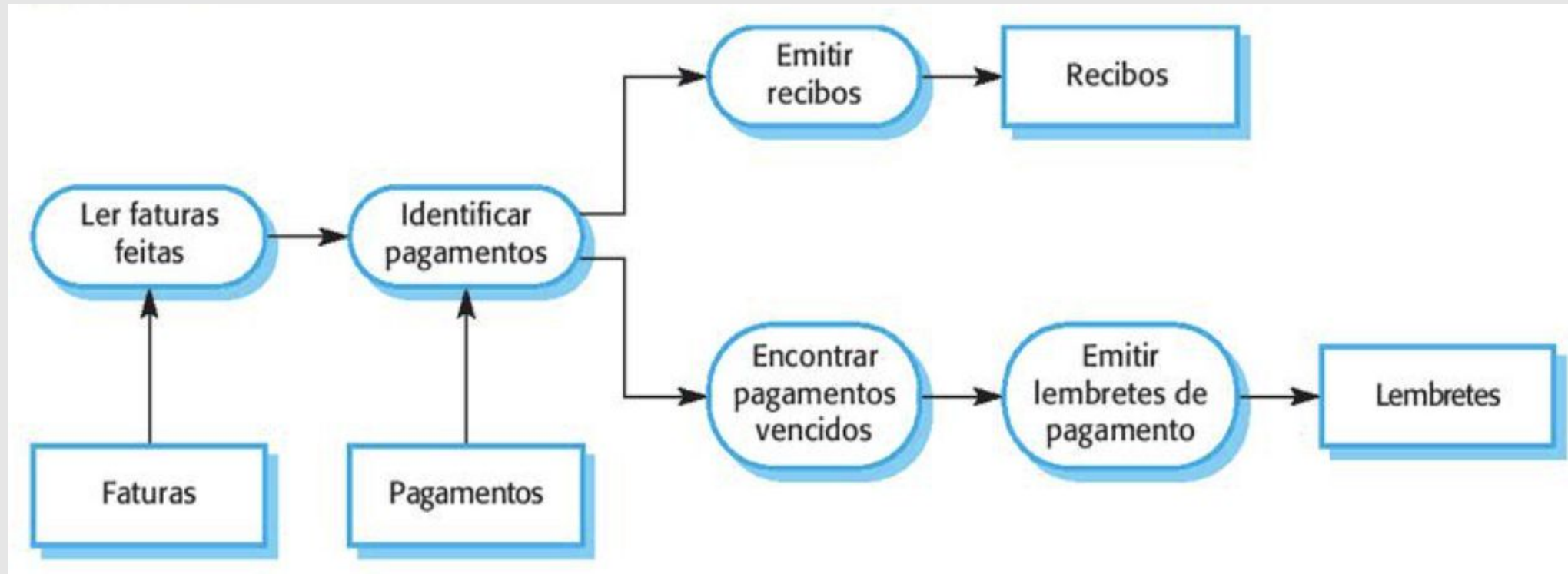
- Fácil de entender e permite o reúso de transformações;
- Evolução por meio da adição de transformações é direta;
- Sistema sequencial ou concorrente.

Arquitetura em Duto e Filtro

- **Desvantagens**

- O formato dados tem de ser acordado;
- Todos aguardam o resultado gerando sobrecarga;
- Impossível reusar componentes arquiteturais que usam estruturas de dados incompatíveis.

Arquitetura em Duto e Filtro



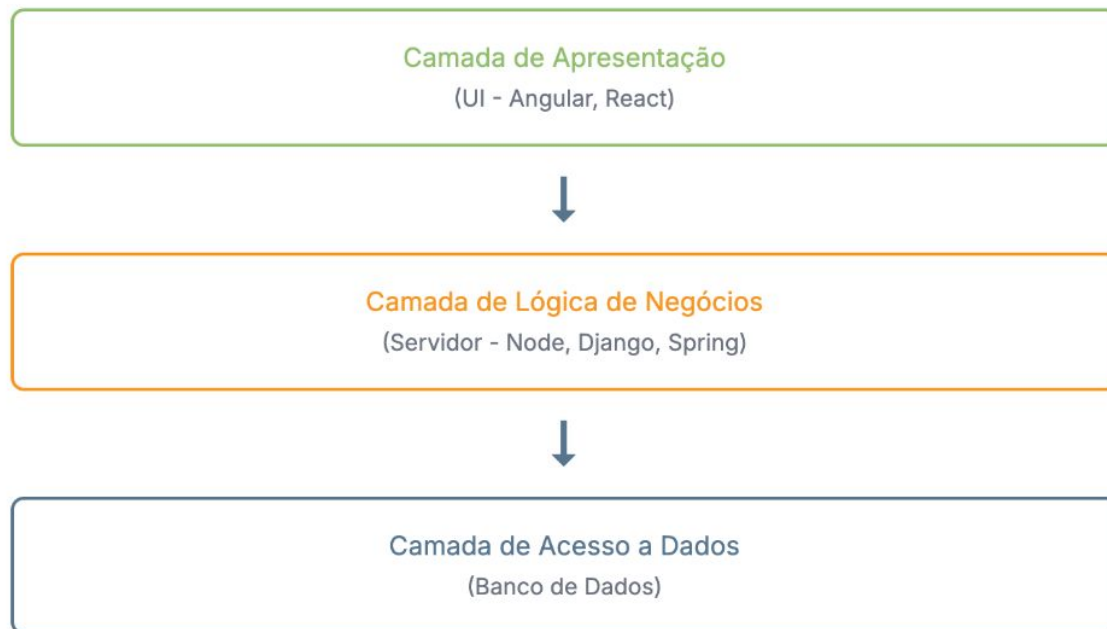
Fonte: Sommerville, 2018, pág.163.

Frameworks/Tecnologias Relevantes

- **Arquitetura em Camadas e Cliente-Servidor**
 - Node.js + Express;
 - Django (Python);
 - Spring Boot (Java).
 - Angular;
 - React;

Frameworks/Tecnologias Relevantes

Arquitetura em Camadas



Modelo Cliente-Servidor



Fonte: Autor

Frameworks/Tecnologias Relevantes

Frameworks Backend

Requisição do Cliente



Servidor de Aplicação
Node.js / Django / Spring Boot



Banco de Dados

Frameworks Frontend

Servidor envia HTML/JS/CSS



Browser executa App
Angular / React

Comunicação via API

Servidor de API

Fonte: Autor

Frameworks/Tecnologias Relevantes

- **Arquitetura em Repositório**

- ETL tools: Apache NiFi, Talend, Pentaho;
- Data lakes: Hadoop, AWS S3;
- SGBDs centralizados: PostgreSQL, Oracle, MongoDB;
- Redis (repositório em memória para dados temporários e rápidos);
- RabbitMQ (fila de mensagens acionadas por eventos).

Frameworks/Tecnologias Relevantes

ETL Gráfico (NiFi, Talend)

Fonte de Dados
(API, DB, Arquivo)



Processadores / Componentes
(Arrasta e solta: Filtra, Enriquece)



Destino Final
(Data Lake, Warehouse)

Data Lake (Hadoop/S3)

Fontes Diversas
(Logs, IoT, Batch)



Repositório Centralizado
(HDFS / S3 Bucket)



Análise e Processamento
(Spark, Athena, ML)

SGBD Relacional (SQL)

Aplicação Cliente



Query SQL
(SELECT * FROM ...)



Dados em Tabelas
(Linhas e Colunas)

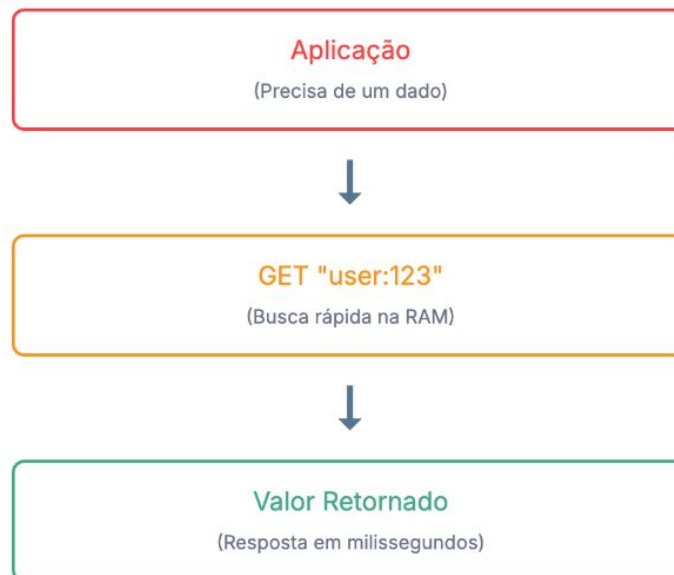
Fonte: Autor

Frameworks/Tecnologias Relevantes

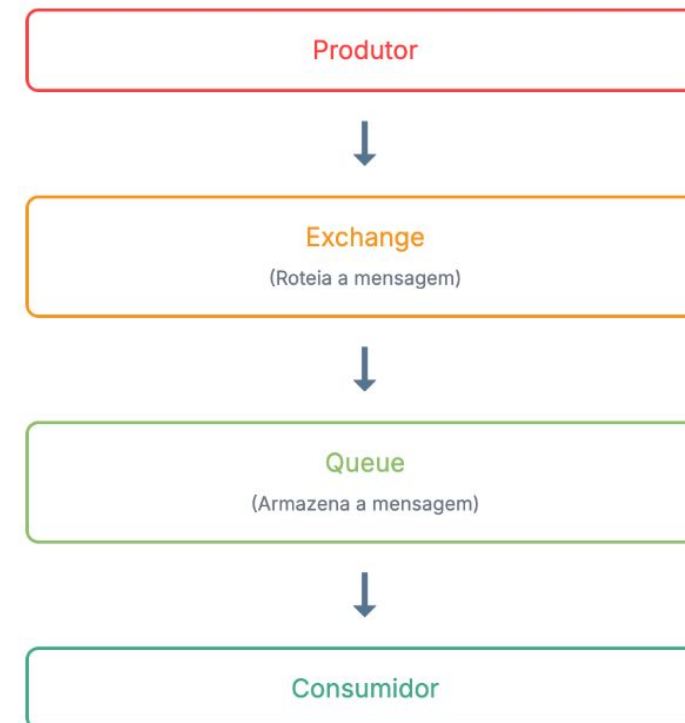
SGBD NoSQL (MongoDB)



Redis (Cache em Memória)



RabbitMQ (Fila de Mensagens)



Fonte: Autor

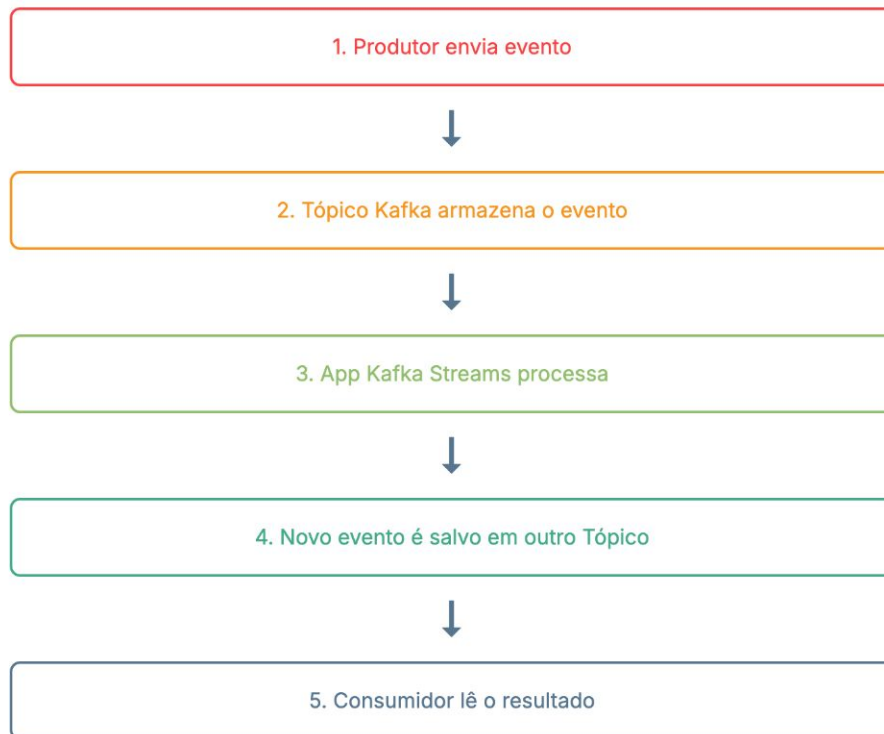
Frameworks/Tecnologias Relevantes

- **Arquitetura em Duto e Filter**

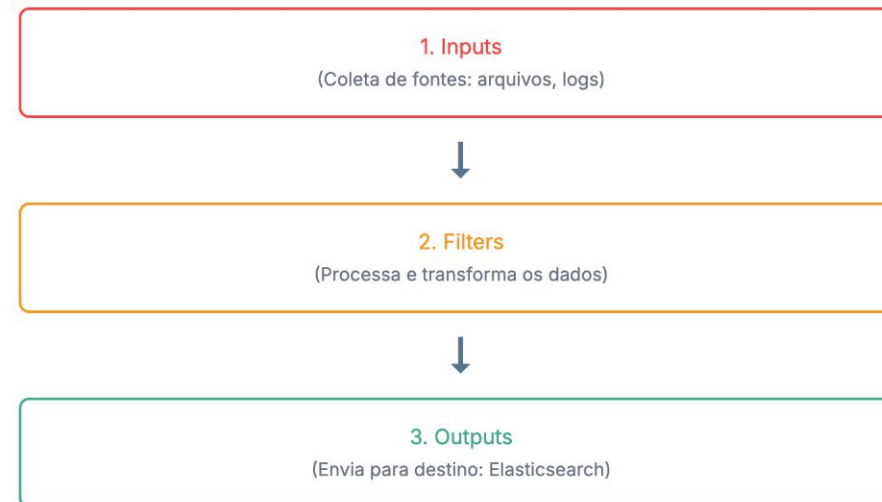
- Apache Kafka + Kafka Streams – fluxo contínuo de dados com transformações;
- Logstash (ELK Stack) – coleta, processa e envia logs por pipelines;
- Unix Pipes + Bash Scripts – paradigma clássico de duto;
- Node.js Streams API – leitura e escrita em fluxo.

Frameworks/Tecnologias Relevantes

Apache Kafka + Streams



Logstash (ELK Stack)



Fonte: Autor

Frameworks/Tecnologias Relevantes

Node.js Streams API

1. Readable Stream

(Lê dados de uma fonte)

`.pipe()`

2. Transform Stream

(Modifica os dados no caminho)

`.pipe()`

3. Writable Stream

(Escreve os dados em um destino)

Unix Pipes + Bash

Comando 1

A Saída Padrão (STDOUT)...

| ...é conectada (pipe) à...

Comando 2

...Entrada Padrão (STDIN) do próximo comando.

```
cat access.log | grep "404" | uniq -c
```

Fonte: Autor

Referências



SOMMERVILLE, Ian. **Engenharia de software**. 10. ed. São Paulo, SP: Pearson, 2018. E-book. Disponível na Biblioteca Digital da UFMS.

Licenciamento



Respeitadas as formas de citação formal de autores de acordo com as normas da ABNT NBR 6023 (2018), a não ser que esteja indicado de outra forma, todo material desta apresentação está licenciado sob uma [Licença Creative Commons - Atribuição 4.0 Internacional](https://creativecommons.org/licenses/by/4.0/).

